# AUTOMATED REAL-TIME SCHEMA DRIFT DETECTION AND ADAPTATION FRAMEWORK FOR STRUCTURED AND SEMI-STRUCTURED DATA

Weerakoon W.R.W.M.N.S.

(IT22204752)

Dissertation submitted in partial fulfillment of the requirements for the
Bachelor of Science (Hons) in Information Technology
Specializing in Data Science

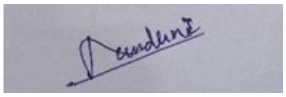
Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

August 2026

# DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute my dissertation, in whole or in part, in print, electronic, or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

| NAME | STUDENT ID | SIGNATURE | DATE |
|---|---|---|---|
| Weerakoon W.R.W.M.N.S. | IT22204752 |  | 22 / 04 / 2026 |

Name of the Supervisor : Dr. Lakminini Abeywardhana
Name of the Co-Supervisor : Dr. Mahima Weerasinghe

The above candidate has carried out research for the Bachelor's degree dissertation under my supervision.

Signature: _____
Supervisor : Dr. Lakminini Abeywardhana Date: 22 / 04 / 2026

Signature: _____
Co-Supervisor : Dr. Mahima Weerasinghe Date: 22 / 04 / 2026

# DEDICATION

*This work is dedicated to my family, whose unwavering love, patience, and sacrifices have been the bedrock of everything I have achieved. Their constant encouragement gave me the strength to persevere through every challenge this journey presented.*
*I also dedicate this work to my academic supervisors and mentors, whose expert guidance and sincere commitment to nurturing research have shaped both this dissertation and my growth as a researcher.*
*Finally, this work is dedicated to the broader data engineering and governance community to those who strive to build more reliable, transparent, and trustworthy data systems. It is my hope that this contribution, however small, moves that effort meaningfully forward.*

## ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Dr. Lakminini Abeywardhana and Dr. Mahima Weerasinghe of the Faculty of Computing, Sri Lanka Institute of Information Technology, for their continuous guidance, constructive feedback, and encouragement throughout this research. Their expertise shaped the scope and quality of this work considerably.

I also wish to thank the panel members for their thoughtful comments and recommendations, which strengthened the technical rigour of the dissertation.

Finally, I extend my appreciation to colleagues and peers who offered support and advice at various stages of this research. Their contributions are genuinely valued.

# ABSTRACT

Schema drift is a persistent and often silent problem in production data pipelines. When upstream schemas change without coordination, downstream processing breaks, analytical outputs become unreliable, and compliance records become incomplete. Existing tools either block all changes rigidly or absorb them silently, and neither approach provides a governed, traceable response.

This dissertation presents a framework that addresses schema drift through four integrated stages: column name normalization, structural change detection, probabilistic rename resolution, and human-in-the-loop governed routing. The rename resolution component combines eight features across lexical, semantic, structural, and statistical evidence dimensions to distinguish genuine column renames from spurious add-remove pairs. Operational risk is assessed through two complementary scoring models: a dataset-level weighted formulation for fast event-level assessment, and a field-level union-of-risks model for granular field-importance-aware scoring. All drifted batches are held in a staging area for human review regardless of risk score, with approve, reject, and rollback actions recorded in an append-only audit log.

The rename resolution model was evaluated on a synthetic corpus of 160 schema-drift scenarios spanning five data domains. Under a scenario-level train-test split requiring generalization to unseen schema structures, the model achieved a Precision-Recall Area Under the Curve of 0.9226 and an F1-score of 0.85. These results demonstrate that rename-aware, risk-informed drift handling produces more reliable and auditable schema adaptation than detection-only or fully autonomous approaches.

**Keywords: Schema Drift Detection, Streaming Data Pipelines, Rename Detection, Risk Scoring, Human-in-the-Loop Governance, Data Observability, Schema Registry.**

# TABLE OF CONTENTS

1. INTRODUCTION ..... 1
1.1 Background ..... 1
1.2 Literature Review ..... 2
1.2.1 Introduction ..... 2
1.2.2 Schema evolution theory and change taxonomies ..... 2
1.2.3 Schema drift detection in practice ..... 4
1.2.4 Schema matching and rename detection ..... 5
1.2.5 Governance, accountability, and human-in-the-loop design ..... 6
1.2.6 Synthesis and research gap ..... 7
1.3 Research Problem ..... 10
1.4 Research Objectives ..... 11
1.4.1 General objective ..... 11
1.4.2 Specific objectives ..... 11
2. METHODOLOGY ..... 13
2.1 Framework Architecture ..... 13
2.2 Stage 1 - Schema Drift Detection ..... 14
2.2.1 Column name normalization ..... 14
2.2.2 Observed schema construction ..... 15
2.2.3 Structural change identification ..... 15
2.3 Stage 2 - Rename Resolution ..... 16
2.4 Stage 3 - Risk Scoring ..... 17
2.4.1 Shared severity model ..... 17
2.4.2 Option A: Dataset-level scoring ..... 19
2.4.3 Option B: Field-level scoring ..... 20
2.5 Stage 4 - Governance and Data Routing ..... 21
2.5.1 The unconditional staging rule ..... 21
2.5.2 Reviewer actions ..... 21
2.5.3 Schema registry and audit log ..... 22
2.6 Commercialization Aspects ..... 22
2.7 Testing and Implementation ..... 23
3. RESULTS AND DISCUSSION ..... 25

3.1 Experimental Setup ..... 25
3.1.1 Evaluation corpus ..... 25
3.1.2 Feature representation ..... 25
3.1.3 Train-test protocol ..... 26
3.2 Results ..... 26
3.2.1 Rename detection performance ..... 26
3.3 Discussion ..... 27
3.3.1 Contribution of each evidence group ..... 27
3.3.2 Operational consequences of rename resolution quality ..... 28
3.3.3 Generalization across domains ..... 29
3.3.4 Compounding integration effects across pipeline stages ..... 29
3.3.5 Governance design justification ..... 30
3.4 Summary of Student Contribution ..... 30
4. CONCLUSION ..... 32

## LIST OF FIGURES

Figure 2.1 Four-stage schema drift detection and adaptation pipeline

Figure 2.2 Severity ordering and risk score computation for Option A and Option B

## LIST OF TABLES

Table 1.1 Literature review summary

Table 3.1 Evaluation corpus domain composition

Table 3.2 Feature groups in the rename detection model

Table 3.3 Train-test protocol summary

Table 3.4 Rename detection model comparison

# LIST OF ABBREVIATIONS

| Abbreviation | Full Form |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| AUC | Area Under the Curve |
| CDC | Change Data Capture |
| CI/CD | Continuous Integration / Continuous Deployment |
| ETL | Extract, Transform, Load |
| F1 | F1-Score (Harmonic Mean of Precision and Recall) |
| HITL | Human-in-the-Loop |
| IoT | Internet of Things |
| JSON | JavaScript Object Notation |
| LLM | Large Language Model |
| ML | Machine Learning |
| NoSQL | Not Only Structured Query Language |
| PII | Personally Identifiable Information |
| PR-AUC | Precision-Recall Area Under the Curve |
| SQL | Structured Query Language |
| XML | eXtensible Markup Language |

# 1. INTRODUCTION

## 1.1 Background

Production data pipelines continuously ingest data from relational databases, event streams, application programming interfaces, and semi-structured sources such as JSON documents and log files. This architectural diversity creates a structural dependency at every stage of the pipeline: downstream components assume that incoming data will conform to the schema they were built against. In practice, this assumption is routinely violated. Columns are renamed during refactoring, fields are removed without prior notice, data types change silently during system migrations, and new attributes appear as business requirements shift [1]. When any of these changes arrive at a downstream system without coordination, the consequences range from broken transformation logic and corrupted analytical outputs to silent compliance failures that do not surface until an audit.

This class of problem is known as schema drift. It differs from planned schema evolution in a fundamental way: planned evolution is coordinated, versioned, and communicated across the pipeline; schema drift is not. It occurs outside any controlled change process, which makes it both harder to detect and harder to respond to in a principled way [1], [2]. A renamed column arrives downstream looking like a missing field. A numeric column serialised as text passes ingestion without error but fails in any downstream arithmetic operation. A deleted regulated field leaves governance reports silently incomplete. The difficulty is not simply noticing that something changed; it is determining what the change means operationally and deciding what to do about it.

Current approaches handle this problem incompletely. Strict schema validators stop failures early but make pipelines fragile, since any legitimate upstream change triggers a halt. Permissive ingestion defers the problem until structural changes have propagated through storage and into analytical layers, where root-cause diagnosis becomes difficult. What production pipelines actually require is a governed layer between ingestion and promotion that can identify what changed structurally,

determine how operationally dangerous that change is, and enforce a traceable human decision before drifted data reaches production [4], [5].

This dissertation presents a framework that provides exactly that layer. The framework integrates four capabilities that have not previously been combined in a single operational runtime service: deterministic column normalization that eliminates false drift alerts before any detection logic runs; multi-feature rename resolution that uses lexical, semantic, structural, and statistical evidence to distinguish genuine renames from spurious add-remove pairs; dual-mode risk scoring that connects detected drift to calibrated operational severity; and governed routing with full auditability through staging, human approve, reject, and rollback decisions, schema version management, and an append-only audit log.

## 1.2 Literature Review

### 1.2.1 Introduction

The management of schema drift in data pipelines sits at the intersection of three established research areas: schema evolution theory, automated schema matching, and data governance. Each area has produced substantial independent contributions. However, as this review demonstrates, these contributions have developed in isolation, and no prior work integrates them into a single operational framework capable of detecting schema drift, resolving rename ambiguity, assessing operational risk, and enforcing a traceable human governance decision before production promotion. This section surveys the key works in each area, identifies points of agreement and disagreement between them, and culminates in a precise statement of the research gap this dissertation addresses.

### 1.2.2 Schema evolution theory and change taxonomies

The earliest challenge in schema drift research was establishing a principled taxonomy of structural change types and their relative operational severity. Without a shared vocabulary for what kinds of changes can occur and what consequences each type carries, neither detection nor risk assessment is possible in a rigorous way.

Chillon et al. [1] addressed this foundational problem by constructing the Orion language on top of a unified schema model spanning both NoSQL and relational databases, verifying a comprehensive taxonomy of schema change operations using the Alloy model checker. Their work establishes that schema changes differ substantially in operational impact: some, such as field deletions, break every downstream consumer that depends on the removed field, while others, such as field additions, extend the schema without affecting existing consumers at all. This ordered view of change impact is the most significant theoretical contribution in the field and directly informs the severity scale applied in the risk engine of this dissertation.

The same research group extended this perspective in [2] by introducing relational concept stability, which tracks whether a schema element retains its semantic identity across repeated comparisons over time. Where [1] addresses the static question of what kinds of changes exist, [2] addresses the dynamic question of how schema identity evolves longitudinally. A field that is stable across many pipeline batches and then suddenly disappears carries different operational weight than a field that has changed repeatedly. The schema registry in this dissertation supports this longitudinal perspective by addressing each schema version through a content hash and preserving the complete version history for rollback and comparison.

Chitnis and Tewari [11] approached schema monitoring from an empirical direction, demonstrating through systematic experiments that hybrid approaches combining statistical profiling with machine learning substantially outperform rule-based threshold monitoring. Rule-based approaches produce systematic false positives when naming conventions vary across source systems and systematic false negatives when type regressions occur within an acceptable field name. This finding directly motivates the multi-feature design of the rename model in this dissertation, where distributional profile features are included precisely because name-based features alone cannot provide reliable discrimination.

Taken together, [1], [2], and [11] establish the theoretical and empirical foundation for what a schema drift detection framework must do: classify changes by type, assess them by operational severity, monitor them longitudinally, and combine multiple evidence sources to do so reliably. What they do not provide is an operational runtime system that acts on these principles.

### 1.2.3 Schema drift detection in practice

Building on the theoretical foundations above, several researchers have attempted to operationalize schema drift detection. These practical contributions differ substantially in scope, autonomy assumptions, and governance posture.

Chintakindhi [4] built an AI-based detection pipeline for schema drift in cloud migration contexts, reporting a 30% reduction in compliance errors relative to manual monitoring. That result establishes that machine learning-based detection outperforms manual review at scale. However, Chintakindhi's framework treats detection as the terminal step: once drift is flagged, no structured response mechanism is defined. The gap between flagging a change and governing what happens to the data as a result of that change is left entirely to the downstream organization. Even a system with high detection accuracy does not, by itself, prevent drifted data from reaching production.

Vaddepalli [5] took a fundamentally different approach with AutoSchema, a self-learning framework that not only detects schema drift but automatically adapts the schema in response, achieving 98.3% detection accuracy with sub-second adaptation latency. Compared to Chintakindhi's detection-only architecture [4], AutoSchema closes the loop between detection and response. However, in regulated pipeline environments such as financial services, healthcare, and government, every schema change requires an explicit, named decision record [12]. A system that applies schema changes automatically cannot satisfy this requirement regardless of how accurate its detection component is. AutoSchema is efficient for low-stakes internal pipelines but unsuitable for regulated contexts. This tension between automation efficiency and governance accountability is a central unresolved conflict in the schema drift literature.

Sirimalla [6] validated the schema registry and version-comparison architecture that underpins schema management in several existing tools, demonstrating applicability in CI/CD pipeline contexts. Sirimalla's contribution is architectural rather than algorithmic: it establishes that content-hash-based schema versioning is operationally sound and that rollback to a prior schema state is a tractable engineering problem. However, Sirimalla does not integrate the registry with a drift detection engine or a risk scoring layer; the registry exists as a standalone versioning tool rather than part of a governed drift response pipeline.

Examining these three contributions together reveals a consistent pattern: each addresses one part of the drift response problem in isolation. Chintakindhi [4] provides detection without response; Vaddepalli [5] provides automated response without governed accountability; Sirimalla [6] provides versioned schema management without detection or risk assessment. The integration of all three capabilities into a single operational workflow is the architectural contribution of this dissertation.

### 1.2.4 Schema matching and rename detection

A persistent limitation of schema drift detection systems, including those described above, is their treatment of column renames. When a column is renamed, a naive structural comparison produces two entries: one deletion and one addition. Without a rename resolution step, the deletion is assigned the highest possible severity in any risk model, causing routine refactoring operations to be flagged as high-risk events. Correcting this requires schema matching, which is the problem of determining whether two schema elements from different schema versions represent the same underlying concept.

Rahm and Bernstein [3] produced the foundational survey of automatic schema matching, classifying existing approaches across four dimensions: schema-level matching, instance-level matching, element-level matching, and structure-level matching. Their taxonomy establishes that no single matching dimension is sufficient in isolation: schema-level features miss vocabulary-divergent renames, instance-level features require data access that may not always be available, and structure-level features are sensitive to schema reorganization. Robust rename resolution therefore requires evidence from multiple dimensions simultaneously, which is the principle that motivates the multi-feature vector design of the rename classifier in this dissertation.

Asif-Ur-Rahman et al. [7] tested this multi-evidence principle empirically in the domain of vegetation data integration, building a hybrid schema matching framework that combines schema-level features with instance-level statistical similarity. Their results confirmed and quantified Rahm and Bernstein's theoretical prediction: hybrid approaches achieve substantially higher matching accuracy than either evidence source alone. This directly informs the inclusion of four statistical features in the

rename model of this dissertation. The limitation of Asif-Ur-Rahman et al.'s work is that it was evaluated on static domain data and was not designed for real-time pipeline contexts.

Priya and Thilagam [8] approached schema matching from a different direction, demonstrating that field semantics encoded as dense sentence embeddings enable substantially better schema grouping than string similarity measures. A rename from transaction_amount to payment_value, for example, contains no shared tokens and would be invisible to a lexical matcher, but the sentence embeddings of natural-language descriptions of both fields would be close in embedding space. This finding establishes semantic similarity as a necessary rather than optional component of any high-recall rename detector.

Sheetrit et al. [9] advanced the state of the art with ReMatch, achieving top-tier schema matching performance by combining embedding-based retrieval with large language model reranking. Compared to [7] and [8], ReMatch is more accurate but substantially more computationally expensive per candidate pair. The tradeoff between accuracy and inference latency is the primary reason ReMatch is identified as a future upgrade path in this dissertation rather than the current implementation.

The schema matching literature converges on a clear conclusion: robust rename detection requires lexical, semantic, structural, and statistical evidence together, with semantic similarity being the single most impactful feature type for recall. What the matching literature does not address is how to integrate rename detection into a governed pipeline workflow where the outcome of each rename decision has direct consequences for risk scoring, staging decisions, and audit log entries.

### 1.2.5 Governance, accountability, and human-in-the-loop design

The question of what to do once schema drift is detected, and who is accountable for that decision, is addressed most directly in the data governance and human-in-the-loop literatures.

Abraham et al. [12] provide the most comprehensive conceptual framework for data governance, defining it through three components: authority over data assets, accountability for data decisions, and mechanisms for making and recording those

decisions. Their framework establishes that governance is not a property of the data itself or of the technical system that processes it, but a property of the decision-making process that surrounds the system. A schema change is governed not when it is detected or scored, but when a named decision-maker with authority over the affected data asset reviews the change, makes an explicit decision, and that decision is recorded with a timestamp and rationale. This principle directly motivates the unconditional staging gate, the three-action reviewer workflow, and the append-only audit log in this dissertation. Abraham et al.'s framework is, however, purely conceptual: it defines what governance requires but does not implement an operational runtime system that enforces it.

Bontempelli et al. [13] approached the human-in-the-loop question empirically, demonstrating that automated systems face a fundamental accuracy ceiling when dealing with ambiguous drift that cannot be safely resolved from the available data features alone. Their finding is that human review is not merely a regulatory convenience but a technical necessity: for a non-trivial fraction of drift events, the structural features visible to an automated classifier do not contain sufficient information to make a safe autonomous decision. This result directly contradicts the design philosophy of AutoSchema [5], which assumes automation alone is sufficient. The unconditional staging gate in this dissertation resolves this conflict by applying human review to all events while using risk scoring to prioritize reviewer attention, rather than attempting to identify in advance which events require human review.

### 1.2.6 Synthesis and research gap

The literature surveyed above establishes five individual findings that together define what an adequate schema drift management framework must do. Chillon et al. [1] establish that schema changes differ in operational severity and must be classified before risk can be assessed. Rahm and Bernstein [3] and subsequent matching researchers [7], [8], [9] establish that rename resolution requires multi-dimensional evidence and that semantic similarity is indispensable for high recall. Chitnis and Tewari [11] demonstrate empirically that hybrid multi-feature approaches substantially outperform rule-based monitoring. The broader concept drift literature

[14], [15] further confirms that monitoring evolving systems requires adaptive, multi-signal approaches rather than static threshold rules. Abraham et al. [12] establish that governance requires named, recorded, and accountable human decisions. Bontempelli et al. [13] establish that human review is a technical necessity for ambiguous events, not merely a regulatory formality; and Yang et al. [16] reinforce this by showing that feature-level explainability is essential for actionable drift management.

What the literature does not provide is a framework that satisfies all five requirements simultaneously in a single operational runtime service. Existing detection systems [4], [5] do not resolve renames before scoring risk, causing routine refactoring to be systematically overscored. Existing matching systems [7], [8], [9] are not integrated into governed pipeline workflows where rename decisions have direct consequences for staging and audit records. Existing governance frameworks [12] are conceptual rather than operational. While the broader drift literature has advanced explainability [16] and stream-native detection [15], these contributions address model-level drift rather than structural schema drift in a governed pipeline context. No prior work combines detection, rename-aware structural comparison, multi-factor risk scoring, and a governed human response into a single runtime workflow. This gap is the precise motivation for the framework presented in this dissertation.

Table 1.1 presents a structured summary of the literature reviewed, mapping each contribution to the evidence it provides and the gap it leaves that the present work addresses.

**Table 1.1: Literature review summary**

| Reference | Key Contribution | Gap Addressed by This Work |
|---|---|---|
| Chillon et al. [1] | Unified schema evolution taxonomy for NoSQL and relational databases; Alloy-verified change classification | Covers planned evolution only; no runtime drift detection or governed response |
| Chillon et al. [2] | Relational concept stability tracking across repeated schema comparisons | Longitudinal tracking without risk scoring or human review integration |

| Reference | Key Contribution | Gap Addressed by This Work |
|---|---|---|
| Rahm and Bernstein [3] | Four-dimension taxonomy for automatic schema matching | Survey classification only; no operational implementation or risk assessment |
| Chintakindhi [4] | AI-based drift detection; 30% compliance error reduction | Detection is the terminal step; no rename resolution, risk scoring, or governed response |
| Vaddepalli [5] | AutoSchema: 98.3% detection accuracy with autonomous adaptation | Fully autonomous; unsuitable for regulated pipelines requiring explicit decision records |
| Sirimalla [6] | Schema registry and version-comparison validation in CI/CD contexts | Registry design validated but not integrated with detection or risk-scored governance |
| Asif-Ur-Rahman et al. [7] | Hybrid schema-level plus instance-level matching outperforms either alone | Applied to static domains only; no real-time pipeline integration |
| Priya and Thilagam [8] | Sentence embeddings enable better schema grouping than string similarity | Clustering application; not combined with lexical or statistical evidence in a rename classifier |
| Sheetrit et al. [9] | ReMatch: state-of-the-art matching via LLM retrieval and reranking | High inference cost; not integrated into a governed pipeline workflow |
| Chitnis and Tewari [11] | Hybrid statistical and ML monitoring outperforms rule-based thresholds | Detection focus only; no rename resolution, risk scoring, or governance workflow |
| Abraham et al. [12] | Data governance defined through authority, accountability, and decision traceability | Conceptual framework; requires an operational runtime implementation |
| Bontempelli et al. [13] | HITL intervention is necessary when automated systems face ambiguous drift | Demonstrated for knowledge drift; not implemented in a schema drift pipeline context |
| Hinder et al. [14] | Comprehensive survey of concept drift detection strategies for evolving | Addresses model-level concept drift; does not address |

| Reference | Key Contribution | Gap Addressed by This Work |
|---|---|---|
| | systems; taxonomy of detection approaches and adaptation mechanisms | structural schema drift or governed pipeline response |
| Caruccio et al. [15] | RFD-based approach for concept drift detection in streaming data; demonstrates constraint-driven detection in continuous data flows | Focuses on value-level drift in data streams; does not address structural schema changes or governed human-in-the-loop response |
| Yang et al. [16] | Feature-level explainable approach to detecting and rationalizing concept drift; demonstrates that interpretability of drift alerts substantially improves operational response quality | Addresses feature-level model drift explainability; not applied to structural schema drift or integrated with a governed pipeline workflow |

### 1.3 Research Problem

The challenge of managing schema drift in production data pipelines presents three interconnected technical problems that existing solutions do not address simultaneously.

The first is rename ambiguity, which systematically inflates risk scores. Every column rename produces a deletion entry and an addition entry in a naive structural diff. Because deletions carry the highest severity in any risk model, rename events are consistently overscored even when they represent routine refactoring with no data loss. Without a probabilistic rename resolution step that evaluates multiple evidence dimensions, a framework cannot distinguish routine refactoring from a genuine field deletion. This distinction has direct and quantifiable consequences for downstream risk assessment and reviewer workload [3].

The second problem is that uniform treatment of all schema changes produces miscalibrated governance responses. A type change on a primary key in a regulated dataset and a new optional column in an internal audit log are both drift events, but their operational severity differs significantly. Without a multi-factor risk scoring model that accounts for change type severity, dataset criticality, sensitivity

classification, field role, and rename confidence, risk signals are not actionable and reviewer triage is not meaningful [12].

The third problem is the absence of a governed response lifecycle. Most detection systems produce an alert or a log entry and stop. There is no mechanism for a team to stage drifted data separately from production, assign a named reviewer, record an approve or reject decision with a rationale, or roll back a prior acceptance. Without this lifecycle, detection accuracy is technically interesting but operationally insufficient [12], [13].

These three problems are mutually reinforcing: inaccurate rename resolution inflates risk scores, which degrades reviewer trust, which undermines the value of the governance lifecycle. The research problem is the absence of an integrated, end-to-end framework that simultaneously addresses all three: rename-aware structural comparison, multi-factor risk scoring, and a governed human-in-the-loop response with full auditability.

## 1.4 Research Objectives

### 1.4.1 General objective

To design, implement, and evaluate a real-time schema drift detection and governed adaptation framework for structured and semi-structured data pipelines, capable of automatically identifying structural schema changes, resolving rename ambiguity through probabilistic multi-feature classification, assessing operational risk through dual-mode scoring, and enforcing traceable human-governed decisions before drifted data reaches production.

### 1.4.2 Specific objectives

SO1: To design and implement a schema drift detection engine that normalizes column representations, infers observed schemas directly from incoming batch data, and identifies structural changes, covering additions, removals, type changes, nullability changes, and renames, across relational and semi-structured data sources.

SO2: To design, train, and evaluate a probabilistic rename resolution classifier that combines lexical, semantic, structural, and statistical evidence across an eight-feature vector, distinguishing genuine column renames from spurious add-remove pairs under

a scenario-level train-test protocol that requires generalization to unseen schema structures.

SO3: To design and validate a dual-mode operational risk scoring system, comprising a dataset-level weighted formulation for fast event-level assessment and a field-level union-of-risks formulation for granular field-importance-aware scoring, connecting detected schema drift to interpretable, calibrated risk levels.

SO4: To design and implement a human-in-the-loop governance module that enforces unconditional staging of all drifted batches for human review, supports approve, reject, and rollback actions with content-hash-based schema version management, and maintains an append-only audit log recording every material governance event with full context.

SO5: To evaluate the integrated framework across held-out scenarios, reporting precision, recall, F1-score, and PR-AUC for the rename resolution model, and analysing the compounding effect of sequential stage integration on downstream risk assessment accuracy.

# 2. METHODOLOGY

## 2.1 Framework Architecture

The framework is structured as a four-stage operational pipeline: schema drift detection, rename resolution, risk scoring, and governance-based routing. Each stage is designed so that its output is a direct, precise input to the next, and the value of the full pipeline compounds beyond what any individual stage delivers in isolation. Figure 2.1 illustrates the overall pipeline structure.

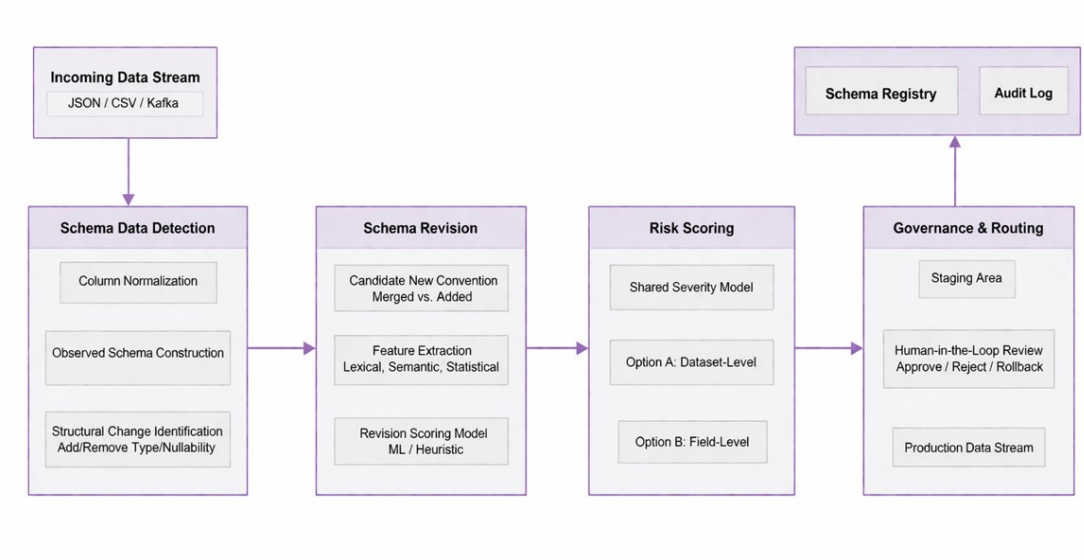



*Figure 2.1: Four-stage schema drift detection and adaptation pipeline*

**Figure 2.1: Four-stage schema drift detection and adaptation pipeline**

Two inputs are provided at dataset registration time. The first is the canonical schema, which is the approved structural reference against which all incoming batches are compared. The second is an optional baseline profile derived from representative historical data, which provides distributional statistics for each registered field and substantially improves rename resolution accuracy when available.

The framework is delivered as a plug-in service that integrates into existing batch or streaming infrastructure without requiring any changes to the underlying pipeline. All state, including the canonical schema, the baseline profile, the schema version history,

and the complete audit log, is maintained internally and is not modifiable by upstream data sources.

## 2.2 Stage 1 - Schema Drift Detection

### 2.2.1 Column name normalization

A significant share of apparent schema drift in production pipelines originates not from semantic change but from naming convention inconsistencies across source systems [11]. The same logical field can arrive as Customer ID from one system, customer_id from another, and customerID from a third. Without normalization, all three appear as different columns and trigger false drift alerts even though the data contract has not changed.

Before any structural comparison, the framework applies a deterministic normalization function to all column names. The transformation applies four rules in order:

- Convert the name to lowercase to eliminate case-style variation.
- Replace all non-alphanumeric characters, including spaces, hyphens, dots, and slashes, with underscores.
- Collapse consecutive underscores into a single underscore.
- Strip leading and trailing underscores from the result.

After this transformation, Customer ID, customer_id, and customer-id all produce customer_id and are treated as identical. The same normalization function is applied at both dataset registration and batch processing time, guaranteeing that all comparisons occur in a shared naming space. This eliminates an entire class of spurious drift events before any detection logic runs. Placing normalization upstream of rename resolution is a deliberate sequencing decision: candidate pairs presented to the rename model are already in a consistent naming space, preventing genuine renames from being penalized by artificial name dissimilarity introduced by source system conventions.

### 2.2.2 Observed schema construction

Once column names are normalized, the system constructs an observed schema directly from the incoming batch data rather than accepting schema declarations from the source system. Source-side declarations are frequently stale or incomplete, and inferring the schema from actual data provides a ground-truth structural snapshot of what has arrived [11].

For each column, the system records three properties: normalized name, inferred data type, and a nullability flag indicating whether any null values appear in the batch. Type inference follows a conservative precedence order: Boolean (checked first to avoid misclassifying 0/1 integer columns), Integer, Float, Numeric string (a string column whose contents are entirely numeric, which indicates a potential silent type regression), Datetime string, and String as the fallback. If any ambiguity exists, the system falls back to the broader type. This prevents false type-change alerts on benign representation differences and ensures only genuine type regressions are flagged.

### 2.2.3 Structural change identification

The observed schema is compared against the canonical schema to produce a raw structural diff. Following the schema change taxonomy of Chillon et al. [1], the detector identifies four direct change categories:

- Additions: columns present in the observed schema but absent from the canonical schema. These extend the data contract without breaking existing downstream consumers.
- Removals: columns expected in the canonical schema but missing from the observed data. These carry the highest inherent risk because every downstream consumer reading a removed field receives null or raises a key error with no safe fallback.
- Type changes: columns present in both schemas with incompatible inferred types. A column transitioning from integer to string passes structural ingestion but silently breaks all downstream operations that depend on type-specific processing. Type regressions are particularly dangerous because the failure surfaces far from the origin.

- Nullability changes: columns where the canonical schema treats the field as non-nullable but the observed batch contains null values. Only the risky direction is flagged; a field becoming stricter than declared is not a risk to downstream consumers.

These four categories form the raw diff. Rename resolution, described in the following section, refines this diff before it is passed to the risk engine, ensuring that risk assessment reflects actual structural changes rather than the naive comparison output.

### 2.3 Stage 2 - Rename Resolution

Without rename detection, every column rename produces a deletion entry and an addition entry in the raw diff. The deletion is assigned severity 1.0 in the risk model, causing rename events to be consistently overscored even when they represent routine field refactoring with no data loss. Rahm and Bernstein [3] identified the distinction between element-level correspondence and apparent structural mismatch as a core challenge in automatic schema matching. The rename resolution component addresses this directly and quantifiably.

For each candidate pair formed by a removed canonical column and an added observed column, the system constructs an eight-feature vector drawing on four evidence dimensions. This multi-source design follows Asif-Ur-Rahman et al. [7], who showed empirically that combining schema-level and instance-level evidence outperforms either source alone:

- Lexical evidence [3]: three features covering character-level edit distance similarity, token overlap after splitting on underscores and camelCase boundaries, and token Jaccard similarity. These features capture straightforward renames such as customer_id to cust_id where surface name similarity is high.
- Semantic evidence [8], [9]: each column is described in natural language from its normalized name, inferred type, and a sample of representative values. That description is encoded with a sentence embedding model and compared using cosine similarity. This captures renames where the underlying concept is

identical but the vocabulary differs entirely, such as transaction_amount to payment_value, which would produce near-zero lexical similarity.

- Structural evidence [3]: type compatibility between the two candidate columns. An exact type match provides strong supporting evidence; a numeric-to-numeric match provides partial credit; an incompatible type pair reduces the rename score.
- Statistical evidence [7], [11]: when a baseline profile is available, four features are computed between the candidate columns: null-rate difference, mean difference, standard deviation difference, and range overlap. Two columns representing the same field under different names will have statistically similar distributions in historical data, providing discrimination that neither lexical nor semantic features can supply for pairs with convergent naming but divergent data behaviour.

The eight features are passed to a trained probabilistic classifier [9] that returns a score in [0, 1] representing the likelihood that the pair is a genuine rename. When no trained model is available, a heuristic weighted combination of the same eight features provides fallback capability. Matching is constrained to be one-to-one. Candidate pairs exceeding the acceptance threshold are accepted as renames, removed from the raw addition and removal lists, and the diff is rewritten to reflect the resolved structural changes before entering the risk engine.

## 2.4 Stage 3 - Risk Scoring

### 2.4.1 Shared severity model

Both scoring options apply a shared severity ordering to each detected change type, grounded in the schema change impact taxonomy of Chillon et al. [1] and ordered by blast radius: the wider the downstream impact of a change reaching production undetected, the higher its assigned severity. Figure 2.2 illustrates the severity ordering and the scoring computation for both options.

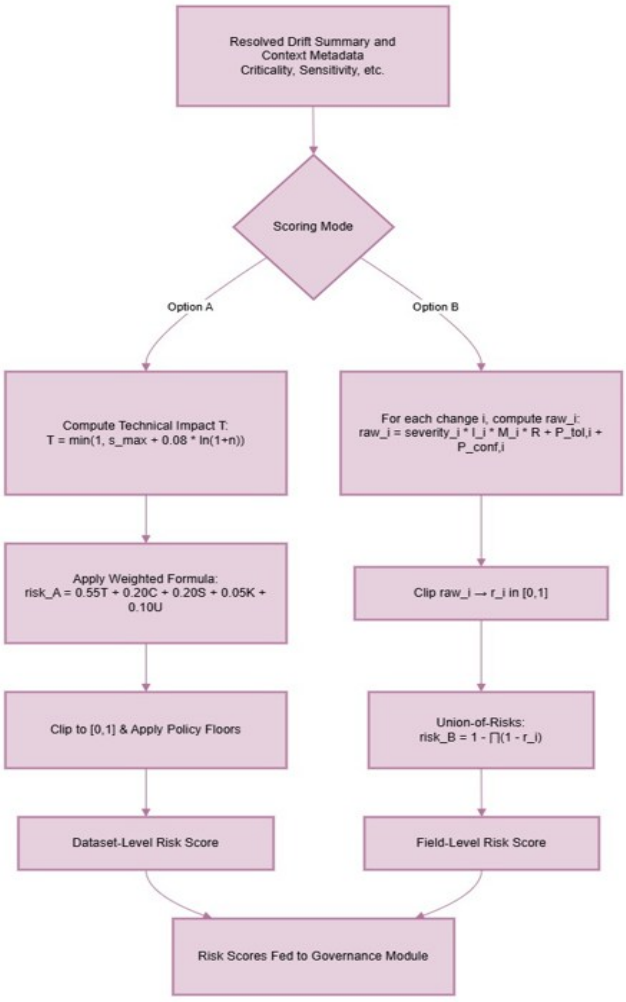



*Figure 2.2: Severity ordering and risk score computation for Option A and Option B*

**Figure 2.2: Severity ordering and risk score computation for Option A and Option B**

- Column deletion (severity 1.0): every downstream consumer reading the deleted column receives null or raises a key error with no safe fallback.
- Type change (severity 0.9): the column name is still present so ingestion succeeds, but every downstream operation depending on the original type produces wrong results or raises exceptions. Type regressions are dangerous because the failure surfaces far from the origin.

- Rename (severity 0.7): the underlying data is present and field lineage is intact, but downstream consumers cannot locate the field by name until the rename mapping is established. Lower severity than deletion reflects that the data is recoverable once the rename is resolved.
- Nullability change (severity 0.5): downstream logic that assumed safe non-null access can produce incorrect results. Flagged only in the risky direction: non-nullable becoming nullable.
- Addition (severity 0.4): new columns extend the schema without breaking existing consumers. The risk is undocumented contract expansion and the introduction of unvalidated fields into regulated pipelines.

### 2.4.2 Option A: Dataset-level scoring

Option A is designed for datasets where a single event-level score is sufficient. The formula has two components: technical impact and business context.

Technical impact T captures the structural seriousness of the drift event:

$$T = \min(1,\ s\_max + 0.08 \times \ln(1 + n)) \qquad \text{(Equation 2.1)}$$

The dominant term s_max is the severity of the most serious change type present in the diff, ensuring that one catastrophic change drives a high T regardless of accompanying minor additions. The logarithmic term adds a small, diminishing increment for each additional affected item n. A logarithm is used rather than a linear term to prevent a large number of low-severity additions from overriding a single high-severity deletion. The min(1,...) cap ensures T never exceeds the normalized maximum.

The final dataset-level score combines technical impact with four business-context factors:

$$risk\_A = clip(0.55T + 0.20C + 0.20S + 0.05K + 0.10U,\ 0,\ 1) \qquad \text{(Equation 2.2)}$$

T carries weight 0.55 because structural severity must dominate regardless of business context. C (weight 0.20) is dataset criticality, configured at registration as Low = 0.3, Medium = 0.6, or High = 0.9. S (weight 0.20) is sensitivity class, configured as None = 0.0, Internal = 0.2, PII = 0.5, or Regulated = 0.8, reflecting compliance and data protection requirements [12]. K (weight 0.05) is a key-field flag set to 1 if any affected

column is a designated business key. U (weight 0.10) is rename uncertainty, computed as one minus the confidence of the lowest-confidence accepted rename, acknowledging that a borderline rename may represent a suppressed genuine deletion. For regulated datasets, policy-based risk floors override the formula: deletions and type changes on strictly regulated datasets are floored at 0.85 regardless of the formula output, reflecting inherent compliance risk that no business-context weighting should minimize [12]. Routing thresholds: score 0.70 or above maps to high risk; 0.35 to 0.70 maps to medium risk; below 0.35 maps to low risk. All three tiers route to staging.

### 2.4.3 Option B: Field-level scoring

Option B is designed for datasets where individual fields carry meaningfully different levels of business importance. A type change on a primary key and a type change on a free-text comment field are structurally identical events but carry very different operational consequences. Option B scores each detected change individually and combines the scores using a union-of-risks formulation [7].

For each detected change i, a raw contribution score is computed:

raw_i = (severity_i x I_i x M_i x R) + P_tol,i + P_conf,i (Equation 2.3)

I_i (Impact) combines dataset criticality d and per-field criticality f, both normalized to [0, 1], as I_i = 0.5d + 0.5f. M_i (Semantic-role multiplier) reflects the structural role of the field: identifiers and primary keys receive 1.0, foreign keys 0.9, timestamps 0.8, measures 0.7, and descriptive dimensions 0.6. R (Sensitivity multiplier) amplifies the entire contribution based on data classification: 1.0 for non-sensitive, 1.05 for internal, 1.15 for PII, 1.25 for lightly regulated, and 1.35 for strictly regulated data [12]. The multiplicative form ensures sensitivity amplifies all risk factors rather than adding a fixed bonus independent of structural severity.

P_tol,i (Tolerance penalty) is added when a change violates a field-level policy: 0.20 for a disallowed removal, 0.18 for a disallowed type change, 0.12 for a disallowed rename, and 0.10 for a disallowed nullability change. P_conf,i (Confidence penalty) applies to rename changes only, computed as 0.15 x (1 - confidence) [9].

Each raw_i is clipped to [0, 1] to produce r_i. The individual scores are then combined:

risk_B = 1 - product_i(1 - r_i) (Equation 2.4)

This formulation captures cumulative risk correctly: two changes each with $r_i = 0.5$ combine to a final score of 0.75 rather than remaining at 0.5. Several moderate changes occurring together are more operationally dangerous than any single moderate change in isolation. Routing thresholds: score 0.80 or above maps to high risk; 0.40 to 0.80 maps to medium; below 0.40 maps to low.

## 2.5 Stage 4 - Governance and Data Routing

### 2.5.1 The unconditional staging rule

The governance rule is unconditional: when no drift is present, data proceeds directly to production; when any drift is detected, the batch is isolated in staging for human review regardless of the risk score. Risk scoring informs the reviewer and prioritizes the review queue, but it does not authorize schema change automatically. This is consistent with the governance principle of Abraham et al. [12] that data decisions must be traceable to named decision-makers with recorded accountability.

The justification for unconditional staging is that risk scoring is calibrated but not certain. A score below the low-risk threshold means the model estimates the event is unlikely to be dangerous given the configured context. It does not mean the event is safe. The semantic content of a schema change, including whether it reflects a business process change, a source system migration, or a data quality regression, is not fully recoverable from structural comparison alone. For regulated datasets, the audit trail is a regulatory requirement: when a downstream compliance failure occurs, the question is who knew about this schema change, when did they know, and what did they decide. A system that auto-approved changes cannot provide a meaningful answer [12].

### 2.5.2 Reviewer actions

Reviewers take one of three actions. Approve and promote activates the candidate schema version in the registry and moves staged data to production storage. Reject preserves the current canonical schema unchanged and marks staged data as rejected; the rejection reason is recorded in the audit log. Rollback reactivates a previously active schema version identified by its content hash, which is useful when a prior

acceptance is found to have caused downstream failures that were not immediately apparent at review time.

This three-action design follows Bontempelli et al. [13] in recognizing that governance over ambiguous structural change requires more than a binary decision. The ability to reverse a previous acceptance is essential for maintaining schema integrity over time.

### 2.5.3 Schema registry and audit log

The schema registry stores versioned schema snapshots identified by content hash, ensuring every schema state is uniquely addressable and reproducible. When drift is detected, the observed schema is stored as a candidate version alongside the existing canonical schema. Approval activates the candidate; rollback reactivates any prior version by hash lookup. This version-controlled architecture mirrors the registry approach validated by Sirimalla [6] in CI/CD pipeline contexts.

The audit log records every material event with full context: drift summary, rename mappings with individual confidence scores, risk score with contributing-factor breakdown, routing decision, and reviewer identity, timestamp, and stated reason for each governance action. The log is append-only, preserving the complete history of how schema state evolved and who was responsible for each transition, supporting post-incident traceability and regulatory audit requirements [12].

## 2.6 Commercialization Aspects

The framework's modular plug-in architecture supports several viable commercial deployment models. As a managed cloud service, the framework would be accessible via REST API and GraphQL endpoints, allowing data engineering teams to register datasets, receive drift alerts, access the governance review interface, and query the audit log without managing infrastructure. Regulated industries, including financial services, healthcare, and government, represent the primary target market because the human-in-the-loop workflow and append-only audit log directly address regulatory requirements for schema change traceability.

As packaged connectors for Apache Kafka, Apache Airflow, dbt, and major cloud data warehouses including BigQuery, Snowflake, and Amazon Redshift, the framework can be distributed through platform marketplaces, enabling integration for teams already operating on these platforms. The dual-mode risk scoring design supports both small teams needing a single dataset-level score through Option A and large enterprises requiring field-level granularity through Option B, broadening the addressable market without requiring separate product configurations.

A tiered subscription model differentiating by dataset volume, schema complexity, and audit retention period is commercially viable. Additional revenue streams include implementation consulting for organizations establishing new data governance programs and domain-specific configuration templates for regulated verticals such as healthcare data pipelines and financial transaction systems.

### 2.7 Testing and Implementation

The framework was implemented as a modular Python service. Each of the four pipeline stages was developed as an independently testable component, enabling component-level unit testing, integration testing of stage-to-stage data flow, and end-to-end scenario testing across the complete pipeline.

The rename resolution component was evaluated on a synthetic corpus of 160 schema-drift scenarios spanning five data domains. The corpus covers abbreviation, token reordering, synonym substitution, case-style variation, and prefix or suffix reformulation, and was constructed to represent the rename types most commonly encountered in production pipelines. From the 160 scenarios, 2,512 candidate column pairs were generated with ground-truth labels.

Evaluation used scenario-level train-test separation rather than pair-level separation. A pair-level split would allow the model to see pairs from the same schema scenario in both training and test sets, inflating recall by exposing domain-specific naming cues that the model should not rely on. Scenario-level separation requires generalization to entirely unseen schema structures. The full model was also evaluated against three progressively stronger baselines, as reported in Chapter 3, to quantify the contribution of each evidence group.

Integration testing confirmed the compounding effect of sequential stage operation described in Section 3.3.4. End-to-end tests confirmed that the complete pipeline produces correct audit log entries, schema registry updates, and staging routing decisions across all representative drift scenario types, including deletions, type changes, renames, nullability changes, and additions.

# 3. RESULTS AND DISCUSSION

## 3.1 Experimental Setup

### 3.1.1 Evaluation corpus

The rename resolution component was trained and evaluated on a synthetic corpus of 160 schema-drift scenarios spanning five data domains, as shown in Table 3.1. From these scenarios, 2,512 candidate column pairs were generated: 819 positive (genuine rename) and 1,693 negative pairs.

**Table 3.1: Evaluation corpus domain composition**

| Domain | Scenarios | Description |
|---|---|---|
| Invoice | 47 | Financial and billing schema variations |
| User Profile | 33 | Account and personal attribute renames |
| Sensor | 32 | Streaming measurement and device fields |
| Retail | 30 | Product, transaction, and inventory fields |
| Geo-Sales | 18 | Sales and location-oriented schemas |
| Total | 160 | 2,512 candidate pairs: 819 positive, 1,693 negative |

### 3.1.2 Feature representation

Each candidate pair is represented by an eight-feature vector drawn from four evidence groups. Table 3.2 describes each group, the features it contributes, and the class of rename it is designed to capture.

**Table 3.2: Feature groups in the rename detection model**

| Evidence Group | Features | Rename Class Captured |
|---|---|---|
| Lexical | Edit distance similarity; token overlap; token Jaccard (3 features) | Surface renames such as customer_id to cust_id |

| Evidence Group | Features | Rename Class Captured |
|---|---|---|
| Semantic | Sentence-embedding cosine similarity over natural-language column descriptions (1 feature) | Vocabulary-divergent renames such as transaction_amount to payment_value |
| Structural | Type compatibility: exact match, numeric-to-numeric, or incompatible (1 feature) | Penalises type-mismatched pairs; reinforces type-consistent renames |
| Statistical | Null-rate difference; mean difference; standard deviation difference; range overlap (4 features) | Distinguishes semantically similar but functionally different fields |

### 3.1.3 Train-test protocol

The 160 scenarios were split at scenario level into 120 training scenarios and 40 test scenarios, yielding 1,824 training pairs and 633 test pairs. The operating threshold was fixed at 0.5359 through precision-recall curve analysis on the training set. Table 3.3 summarizes the protocol.

**Table 3.3: Train-test protocol summary**

| Protocol Item | Value |
|---|---|
| Total scenarios | 160 across 5 domains |
| Train / Test split | 120 / 40 scenarios (scenario-level separation) |
| Training pairs | 1,824 |
| Test pairs | 633 |
| Total candidate pairs | 2,512 (819 positive, 1,693 negative) |
| Operating threshold | 0.5359 (fixed by PR curve analysis on training set) |
| Primary evaluation metrics | Precision, Recall, F1-score, PR-AUC |

## 3.2 Results

### 3.2.1 Rename detection performance

The full model was evaluated against three progressively stronger baselines on 633 held-out test pairs. At the selected operating threshold, the full model achieved 0.80 precision, 0.90 recall, 0.85 F1, and 0.92 PR-AUC with an overall accuracy of 0.8957. Table 3.4 presents the comparison across all configurations.

**Table 3.4: Rename detection model comparison (633 held-out test pairs)**

| Model Configuration | Precision | Recall | F1-Score | PR-AUC |
|---|---|---|---|---|
| Name similarity only (baseline 1) | 0.71 | 0.64 | 0.67 | n/a |
| Lexical features plus structural type (baseline 2) | 0.77 | 0.74 | 0.75 | 0.81 |
| Lexical plus structural plus semantic (baseline 3) | 0.79 | 0.85 | 0.82 | 0.89 |
| Full model: all four evidence groups | 0.80 | 0.90 | 0.85 | 0.92 |

## 3.3 Discussion

### 3.3.1 Contribution of each evidence group

The 18-point F1 improvement from baseline 1 (0.67) to the full model (0.85) is the central empirical result of this evaluation. It demonstrates that rename resolution is not a string comparison problem. The baseline achieves reasonable precision (0.71) but poor recall (0.64) because a large proportion of real-world renames involve no shared surface tokens. Abbreviations such as transaction_amount to tx_amt, synonym substitutions such as unit_price to item_cost, and token reorderings such as date_of_birth to birth_date all produce near-zero lexical similarity scores. The single-feature baseline misses these entirely.

Adding the type compatibility feature (baseline 2) improves both precision and recall. The structural feature provides evidence that is orthogonal to name similarity: it rewards type-consistent pairs and penalises type-incompatible pairs independently of how similar the column names are.

Adding sentence-embedding cosine similarity (baseline 3) produces the single largest performance jump, lifting recall from 0.74 to 0.85. Embedding models encode

semantic relatedness regardless of surface form and capture exactly the vocabulary-divergent renames that lexical features cannot reach [8], [9]. Any rename detection system built solely on name similarity will have a recall ceiling approximately 20 points below what is achievable with semantic evidence.

Adding the four statistical features (full model) improves precision from 0.79 to 0.80. Statistical features address a specific failure mode: pairs that appear semantically similar due to convergent naming but represent genuinely different fields. When two columns have similar names but their null rates, means, and value ranges differ substantially, the statistical features provide a discriminating signal that neither lexical nor semantic evidence can supply [7], [11]. This confirms the design decision to require baseline profile data as a first-class input at dataset registration time.

### 3.3.2 Operational consequences of rename resolution quality

The interaction between rename resolution quality and downstream risk scoring is directly quantifiable. When the model correctly identifies a genuine rename, that pair is removed from the raw diff and scored at severity 0.7. When the model misses a genuine rename, the canonical column remains as a deletion at severity 1.0. For Option A, this inflates the T component: the s_max term increases from 0.7 to 1.0, shifting the risk_A score upward by approximately 0.165 weighted units. On a dataset configured with Medium criticality and Internal sensitivity, this shift is sufficient to move an event from the low-risk tier into the medium-risk tier, triggering a staging review for what is operationally a routine field rename. At scale, across hundreds of daily batches, systematic false escalations of this kind degrade reviewer trust and produce alert fatigue that undermines the governance workflow.

The reverse failure mode, accepting a false rename, is less frequent but more dangerous. If the model incorrectly matches a deleted canonical column to an unrelated added column, it removes a genuine deletion from the diff and the reviewer never sees it. If the event is then approved, the canonical schema is updated without acknowledging that a field has been lost. The rename uncertainty term U in Option A and the confidence penalty P_conf in Option B are designed to mitigate this: both add

residual risk proportional to how uncertain the rename decision was, so that borderline renames near the acceptance threshold do not fully suppress the risk signal.

### 3.3.3 Generalization across domains

Achieving 0.85 F1 under scenario-level train-test separation across five heterogeneous domains is a meaningful result. The scenario-level separation requires the model to generalize to entirely unseen schema structures. The five-domain design ensures the model encounters the naming diversity of production pipelines: sensor field names bear no resemblance to invoice field names, and rename patterns in user profile schemas are structurally different from those in retail inventory schemas. The reported performance reflects genuine generalization rather than within-domain memorization.

### 3.3.4 Compounding integration effects across pipeline stages

The four stages of the framework compound in value when operating in sequence. Three specific effects are worth examining directly.

First, normalization upstream of rename resolution reduces the feature dimensionality the rename model must handle. Without normalization, Customer ID and customer_id would be presented as a near-match pair with moderate edit distance. With normalization, they are presented as the identical string customer_id, eliminating false negatives where genuine renames were penalized by artificial name dissimilarity from source system conventions.

Second, rename resolution upstream of risk scoring ensures the diff entering the risk engine reflects actual structural changes. Without rename resolution, a batch containing three column renames presents as three deletions plus three additions, giving $s_{max} = 1.0$ and $n = 6$. With rename resolution, the same batch presents as three renames at severity 0.7 and $n = 3$. The difference in Option A T-values is not marginal, and the risk score the reviewer receives accurately represents what actually changed.

Third, risk scoring upstream of governance routing means reviewers receive a prioritized queue rather than an undifferentiated list of pending events. A reviewer receiving a score of 0.92 with a factor explanation identifying a deletion of a regulated

PII key field can act immediately. A reviewer receiving a score of 0.28 identifying only a nullable flag change on a non-critical descriptive attribute can schedule that review for a lower-priority window. This prioritization is how risk scoring translates into operational reviewer efficiency.

### 3.3.5 Governance design justification

The unconditional staging gate increases reviewer workload compared to a system that auto-approves low-risk events. This is a deliberate and accepted cost. Risk scoring is calibrated but not certain. A score below the low-risk threshold means the model estimates the event is unlikely to be dangerous given the configured context. It does not mean the event is safe. Schema changes with low structural risk scores can reflect significant business events, including source system migrations or data collection methodology changes, that are not recoverable from structural comparison alone.

For regulated datasets, the audit trail is a regulatory requirement. When a downstream compliance failure occurs, regulators ask who knew about this schema change, when did they know, and what did they decide. A system that auto-approved changes cannot provide a credible answer. A system that required a named reviewer to explicitly approve with a recorded timestamp and stated reason can. The governance lifecycle is the mechanism through which detection becomes operationally accountable [12].

## 3.4 Summary of Student Contribution

This research was conducted as an individual project. The following contributions were made by the student across the full research lifecycle:

- Formulated the research problem and identified the specific gap through systematic review of twelve foundational references spanning schema evolution theory, drift detection, schema matching, and data governance.
- Designed and implemented all four pipeline stages: the normalization engine, the observed schema inference module, the structural diff engine, and the rename resolution pipeline.

- Constructed the 160-scenario synthetic evaluation corpus across five data domains and generated 2,512 candidate column pairs with ground-truth rename labels.
- Engineered the eight-feature vector representation, trained the probabilistic rename classifier under scenario-level separation, and conducted the four-configuration ablation study.
- Designed both Option A and Option B risk scoring models, including the mathematical formulation, weight calibration, policy floor logic for regulated datasets, and routing threshold design.
- Designed and implemented the governance and routing module, the schema registry with content-hash-based versioning, and the append-only audit log with complete event context.
- Conducted all integration analysis quantifying the compounding effect of sequential stage operation and authored all sections of this dissertation.

# 4. CONCLUSION

Managing schema drift in production data pipelines has historically been addressed only in part. Detection tools exist, but detection alone does not prevent drifted data from reaching production, does not explain why a detected change is operationally dangerous, and does not provide a mechanism for a team to make and record a governed decision about a schema change. This dissertation addressed the complete problem.

The framework presented combines column normalization, multi-feature probabilistic rename resolution, dual-mode risk scoring, and governed human review into a single end-to-end workflow delivered as a modular plug-in service. The rename model, evaluated across 160 scenarios with scenario-level train-test separation, achieved 0.85 F1 and 0.92 PR-AUC under the generalization requirement of unseen schema structures. The ablation study showed that semantic evidence is responsible for the single largest recall improvement, lifting recall by 21 points over the lexical-only baseline, and that statistical distributional features improve precision by eliminating false matches between semantically similar but functionally distinct fields.

The risk scoring analysis demonstrated that rename resolution quality has direct and quantifiable consequences for downstream risk assessment: a missed genuine rename inflates the Option A risk score by approximately 0.165 weighted units, sufficient to move a routine refactoring event from the low-risk tier into the medium-risk tier. The confidence penalty terms U in Option A and P_conf in Option B provide residual risk signal proportional to rename uncertainty, ensuring that borderline accepted renames do not fully suppress the risk signal from potentially misclassified deletions.

The broader contribution is a design principle: schema drift should be treated as a governed change event with a complete traceable lifecycle. The staging gate, the approve, reject, and rollback workflow, and the append-only audit log are not operational overhead. They are the mechanism through which detection becomes accountable. By requiring a named reviewer to explicitly approve or reject each schema change with a recorded timestamp and stated reason, the framework transforms an opaque pipeline event into a traceable governance decision that satisfies regulatory audit requirements and supports long-term pipeline integrity.

The following directions are identified for future work:

- Larger evaluation corpus: the 160-scenario corpus establishes proof of concept. A production-ready model requires exposure to schemas with hundreds of columns across a broader range of domain naming conventions. Real-world schema drift logs from industry partners would substantially improve generalization.
- Retrieval-and-reranking rename architecture: replacing the probabilistic classifier with a retrieval-based architecture following Sheetrit et al. [9] would reduce errors in the ambiguous mid-confidence region. Large language model reranking of candidate pairs is the natural upgrade path for the rename component.
- Multi-step governance for regulated datasets: a two-step review workflow covering technical impact assessment followed by compliance officer confirmation would be more defensible in strictly regulated environments. The audit log infrastructure already supports this; the extension is primarily in the routing logic.
- Streaming latency evaluation: a dedicated evaluation measuring end-to-end latency from batch arrival to staging routing decision, under varying batch frequencies and schema complexity levels, would validate the framework's suitability for high-velocity production pipelines.
- Benchmark release: publishing the evaluation corpus with ground-truth rename mappings and candidate pair labels as a public benchmark would enable reproducible comparison between rename detection methods and accelerate progress in the schema drift literature.

# REFERENCES


[1] P. Chillon, M. Klettke, D. S. Ruiz, and J. G. Molina, "A generic schema evolution approach for NoSQL and relational databases," IEEE Trans. Knowl. Data Eng., 2024. https://doi.org/10.1109/TKDE.2024.3362273

[2] A. H. Chillon, M. Klettke, D. S. Ruiz, and J. G. Molina, "Schema drift: Relational concept stability across repeated comparison," unpublished manuscript.

[3] E. Rahm and P. A. Bernstein, "A survey of approaches to automatic schema matching," VLDB J., vol. 10, no. 4, pp. 334-350, 2001.

[4] S. K. Chintakindhi, "AI-driven schema drift detection: Automating regulatory compliance in cloud migration projects," IJIRMPS, vol. 13, no. 2, Mar.-Apr. 2025.

[5] R. K. Vaddepalli, "AutoSchema: A self-learning framework for detecting and adapting to schema drift in real-time data streams," Eur. J. Adv. Eng. Technol., vol. 10, no. 7, pp. 94-100, 2023.

[6] A. Sirimalla, "Automated schema drift detection and resolution in multi-database CI/CD pipelines," Frontiers Comput. Sci. Artif. Intell., 2024.

[7] M. Asif-Ur-Rahman et al., "A semi-automated hybrid schema matching framework for vegetation data integration," arXiv:2305.06528, 2023. https://arxiv.org/abs/2305.06528

[8] D. U. Priya and P. S. Thilagam, "JSON document clustering based on schema embeddings," J. Inf. Sci., Sep. 2022. https://doi.org/10.1177/01655515221116522

[9] E. Sheetrit, M. Brief, M. Mishaeli, and O. Elisha, "ReMatch: Retrieval enhanced schema matching with LLMs," arXiv:2403.01567, Mar. 2024. https://doi.org/10.48550/arXiv.2403.01567

[10] Y. Lee, Y. Lee, E. Lee, and T. Lee, "Explainable artificial intelligence-based model drift detection applicable to unsupervised environments," CMC: Comput. Mater. Contin., vol. 76, no. 2, 2023.

[11] A. Chitnis and S. Tewari, "Detecting data drift and ensuring observability with machine learning automation," JETIR, vol. 9, no. 8, Aug. 2022.

[12] R. Abraham, J. Schneider, and J. vom Brocke, "Data governance: A conceptual framework, structured review, and research agenda," Int. J. Inf. Manag., vol. 49, pp. 424-438, Dec. 2019.

[13] A. Bontempelli et al., "Human-in-the-loop handling of knowledge drift," Data Min. Knowl. Discov., 2022.

[14] F. Hinder, A. Vaquet, and B. Hammer, "One or two things we know about concept drift—a survey on monitoring evolving systems," Front. Artif. Intell., vol. 7, 2024. https://doi.org/10.3389/frai.2024.1330257

[15] L. Caruccio, S. Cirillo, G. Polese, G. Sabbatino, and S. Venticinque, "An RFD-based approach for concept drift detection in data streams," Inf. Sci., vol. 706, 2025. https://doi.org/10.1016/j.ins.2025.121725

[16] L. Yang et al., "Detecting and rationalizing concept drift: A feature-level explainable approach," Expert Syst. Appl., vol. 268, 2025. https://doi.org/10.1016/j.eswa.2025.126320